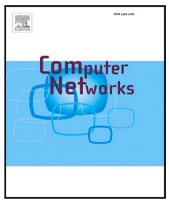
Computer Networks 267 (2025) 111364



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet





P4-Assisted Slowloris DDoS attack detection in IoT environments by using ML and DL

Erick D. Ramirez-Martinez [a], Jesús A. Pérez-Díaz [a,*], Noe M. Yungaicela-Naula [b]

[a] Tecnologico de Monterrey, School of Engineering and Sciences, Monterrey 64849, Mexico
[b] Centre for Secure Information Technologies (CSIT), Queen's University Belfast, Belfast, BT3 9DT, Northern Ireland, UK

ARTICLE INFO

Keywords:
Programming Protocol-Independent Packet Processors (P4)
Slow-rate Distributed denial of service (DDoS)
Internet of Things (IoT)
Machine learning (ML)
Deep learning (DL)
Software defined networking (SDN)

ABSTRACT

Distributed denial of service (DDoS) attacks and their more sophisticated slow-rate variants continue to pose a major threat to next-generation networks, such as internet of things (IoT). Due to their limited computational and storage capacities, IoT devices often lack adequate security protections, leaving them vulnerable. Cybercriminals have exploited this problem to create botnets, which are then used to target vital infrastructures. Recent studies have employed Software-Defined Networking (SDN) and machine learning (ML) to autonomously identify slow-rate DDoS attacks. However, because recent works employ a centralized SDN controller, this strategy causes overload as the network size increases. To reduce the workload on the SDN controller, we propose a Programming Protocol-Independent Packet Processors (P4)-based framework that uses ML or deep learning (DL) to detect and mitigate Slowloris DDoS attacks in IoT networks. Our framework employs P4 programmable switches to collect network traffic characteristics and forward them to an intrusion detection system (IDS) for attack detection. We evaluated the framework using Mininet and BMv2 switches, demonstrating that it can detect Slowloris DDoS attacks with up to 98% accuracy using either of the following models: random forest (RF), k-nearest neighbor (KNN), decision tree (DT), long-short-term memory (LSTM) neural network, convolutional neural network (CNN), gated recurrent (GRU) neural network, and multi-layer perceptron (MLP) models. Mitigation is achieved by modifying the match action tables of the switches to block attacker IPs based on IDS results.

1. Introduction

DDoS attacks aim to deplete the resources allocated to a target service, thereby impeding its use by legitimate users. DDoS attacks are increasing not only in quantity but also in complexity. According to a report from Enisa in [1], 21.4% of cyberattacks performed between July 2022 and June 2023 were DDoS attacks. Attackers have various motivations for performing DDoS attacks, including monetary or political reasons, which further drive the frequency and complexity of such attacks [2].

While flood attacks, such as TCP or SYN floods, remain the most common form of DDoS attack, there are other, more stealthy and complex variations known as slow, slow-rate, low-rate or stealthy DDoS attacks [3–5]. Rather than depleting resources through a large number of service requests, slow-rate attacks retain resources by prolonging the communication as much as possible while sending only a small number of packets. As a result, attacking hosts are often mistaken for legitimate hosts with slow internet connections [3]. Examples of slow-rate attacks include Slow HTTP, Slowloris, and RUDY (Are you dead yet?). It is worth noting that slow-rate DDoS attacks are classified as application layer DDoS attacks, which have seen an exponential increase of 82% in recent years [6].

Furthermore, according to a report by Microsoft [7], there is a growing trend of DDoS attacks targeting IoT devices. IoT devices are vulnerable to security threats due to factors such as weak authentication, lack of encryption, and inadequate integrity protection. These vulnerabilities make IoT devices prime targets for infection and exploitation in launching DDoS attacks [2,8,9]. Even though IoT devices are not powerful enough to perform large scale flood attacks, they can still carry out stealthy slow-rate DDoS attacks that lead to resource depletion. Additionally, IoT devices generally have limited processing capabilities, which often result in lower communication bandwidth. This characteristic makes IoT devices resemble those used in slow-rate attacks, leading attackers to be mistaken for devices with low bandwidth [10].

Several methods have been proposed to detect and mitigate slow-rate DDoS attacks using SDN [5]. The most common approach involves

* Corresponding author.
E-mail address: jesus.arturo.perez@tec.mx (J.A. Pérez-Díaz).

https://doi.org/10.1016/j.comnet.2025.111364
Received 11 April 2024; Received in revised form 16 April 2025; Accepted 5 May 2025
Available online 18 May 2025
1389-1286/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

deploying solutions on the control plane to detect and mitigate network attacks [5]. Security applications deployed in the control plane collect network statistics and perform attack detection using various methods, and some of them involve machine learning (ML) or deep learning models (DL). Once an attack is detected, it is essential for security mechanisms to implement mitigation actions. However, if only a single controller is deployed on the control plane, it may experience overhead due to unusual high traffic conditions [11]. This overhead becomes a significant challenge, particularly in large-scale networks like IoT networks [12]. As a result, a solution that operates in conjunction with the data plane becomes essential. The data plane of SDN can be involved by leveraging devices that support P4. P4 is a programming language designed to offer programmability for compatible network devices.

To enhance the scalability and security of networks with IoT devices, we propose a hybrid framework based on the collaboration between the control plane and P4-programmable switches in the data plane. This framework is proposed to be implemented on P4 switches that handle packet processing and feature calculation at the edge of a network, which offloads the control plane from these tasks. The features calculated by programmable switches are sent to an intrusion detection system (IDS), which utilizes ML or DL to detect Slowloris DDoS attacks. We also propose a mitigation mechanism (intrusion prevention system, IPS) that configures the match action-tables of the P4 switches to block attackers.

Three ML and four DL models are trained using the CICIoT2023 dataset presented by Neto et al. [13] along with a Slowloris tool for real attack traffic generation. Furthermore, a testbed is deployed using Mininet and BMv2 switches to demonstrate the detection performance of the framework in actual implementations.

In summary, our contributions are as follows:

1. We provide a framework that detects and mitigates Slowloris attacks in IoT networks, employing a stateful data plane approach.
2. We provide a comparative analysis of the performance of several ML and DL models in detecting Slowloris DDoS.
3. We provide an evaluation of the framework in terms of detection and response times, employing a testbed in a Mininet simulation.

The rest of the document is organized as follows. Section 2 briefly summarizes previous works. The background knowledge is provided in Section 3. The framework and the structure of the system is described in Section 4. Section 5 presents the experiments performed to evaluate the solution. The discussion is provided in Section 6. Finally, the conclusion and future work are included in Section 7.

2. Literature review

Recently, several studies have taken advantage of programmable switches and external devices in the data plane to detect attacks using ML models. For example, Carvalho et al. [14] proposed a DDoS attack detection method in the data plane by leveraging white-box switches. These switches implement a data plane capable of collecting network information and using registers in the programmable switches to store the information. The information stored in the registers is then retrieved to perform attack detection using either K-nearest neighbors (KNN), support vector machine (SVM) or random forest (RF) models.

In a similar approach, Macias et al. [15] presented ORACLE, a system that detects DDoS attacks by leveraging P4 switches, which capture network information and store it in registers. The stored information is read at time intervals and sent to an SDN controller, which classifies network flows using KNN or RF models.

Another work is presented by Musumeci et al. [16], who proposed DDoS attack detection (DAD) and performed several tests to compare the performance of different feature extraction techniques in P4. DAD was tested with detection performed either on-site (inside the P4 switch) or by delivering information to an external IDS. In the second case, three scenarios were explored: (i) packet mirroring, (ii) header mirroring, and (iii) feature extraction. The authors performed DDoS detection using using KNN, RF, SVM or artificial neural network (ANN) models.

Roshani and Nobakht [17] presented HybridDAD, a framework based on the study presented in [16]. HybridDAD performs attack detection of several flood DDoS attack types using P4 switches. The authors reported a detection accuracy of up to 99% using RF, SVM, KNN, or recurrent neural network (RNN). However, they did not provide a network topology for testing. Additionally, the authors used a dataset created by themselves, which may not be fully representative for actual attack traffic as the attackers may use different parameters than those used by the authors.

The work in [18] proposed 3D-PM, a solution for detecting flood DDoS attacks using KNN, RF, ANN, Naive Bayes or SVM models. Network packets are processed in the data plane using P4 switches to extract features. Features are sent in fixed time windows to an IDS using packets with a custom header. The authors treated the attacks as a multi-classification problem, whereas switching to binary classification might yield better results. For example, SVM models were originally designed for binary classification [19].

Doriguzzi-Corin et al. [20] introduced P4DDLe, a framework that detects DDoS attacks using P4 programming, Bloom filters and LUCID. LUCID is a convolutional neural network (CNN) designed to detect attacks in network environments, while Bloom filters are arrays that use hashing to keep an approximation of the count of items. P4DDLe achieved an accuracy above 99%.

Recently, alternative collaborative systems emerged. These systems distribute and coordinate tasks such as data collection, attack detection, and mitigation between the SDN control plane and the data plane. For example, Sadi et al. [21] proposed a system that coordinates the SDN data and control planes, utilizing artificial intelligence (AI) for DDoS detection. Their system detects attacks by measuring the packet asymmetry of the flows between devices. Packet asymmetry refers to the difference between the number of incoming and outgoing packets. If the asymmetry of a flow exceeds a certain threshold, the data plane collects features from the flow. Flow features are sent to a CNN for classification. If an attack is detected, the control plane updates the switch tables to block the attack. It should be noted that this asymmetry-based detection approach exploits the attack characteristics of flood DDoS and might not be suitable for detecting slow-rate DDoS attacks.

Similarly, Quintero González et al. [22] proposed Bungee-ML, a system based on the collaboration between the data plane and control plane in SDN. Bungee-ML uses entropy analysis, where the frequency of IP addresses is calculated and compared to certain expected bounds to detect attacks. A suspicious list is filled with the IP addresses that exceed the expected bounds during entropy analysis. If the entropy analysis determines a packet is suspicious, the header of the packet is sent to a controller for classification. The controller performs classification by using either a RF or Naive Bayes model. Additionally, this system performs mitigation using a pushback mechanism and suspicious lists. The suspicious lists define how to handle malicious packets, while the pushback mechanism sends the lists to the rest of the data plane. Bungee-ML relies on a CPU mounted in P4 switches to perform complex operations. This requirement results in increased costs if several switches implement a CPU in a network as it requires the installation of additional components to the network devices. Moreover, implementing entropy analysis in IoT networks may be impractical, as IoT devices could be mistakenly identified as attackers [10].

In another approach, Ma et al. [23] proposed a detection and mitigation system based on a graph CNN. The CNN uses P4 telemetry to analyze the state of the network and sends this information to an SDN controller to detect flood DDoS attacks. The SDN controller performs mitigation by applying dynamic lists and rerouting traffic when the number of packets passing through a switch exceeds a certain

Table 1
Our work vs previous research.

Author	High-rate	Slow-rate	IDS	IPS	IoT	ML	DL
Carvalho et al. [14]	✓	–	✓	–	–	✓	–
Macias et al. [15]	✓	–	✓	–	–	✓	–
Musumeci et al. [16]	✓	–	✓	–	–	✓	✓
Roshani and Nobakht [17]	✓	–	✓	–	–	✓	✓
Mecerhed et al. [18]	✓	–	✓	–	–	✓	–
Doriguzzi-Corin et al. [20]	✓	–	✓	–	–	–	✓
Sadi et al. [21]	✓	–	✓	✓	–	–	✓
Quintero González et al. [22]	✓	–	✓	✓	–	✓	–
Ma et al. [23]	✓	–	✓	✓	–	✓	✓
Friday et al. [10]	✓	✓	✓	✓	✓	–	–
Kapourchali et al. [24]	–	✓	✓	✓	–	✓	✓
Our work	–	✓	✓	✓	✓	✓	✓

threshold. The mitigation strategy involves rerouting malicious packets to a specific location. The authors use a packet number threshold to determine an attack, which may not be effective in detecting slow-rate attacks. Flood DDoS attacks generate a high number of packets, which exceed the threshold of the proposed solution; however, that may not be the case for slow-rate attacks, which generate a small number of packets that may not exceed the threshold.

Friday et al. [10] proposed a solution for slow-rate DDoS attacks. These authors also included the study of SYN, TCP, ICMP, and UDP flood attacks. Their solution uses a decision tree to identify IoT devices (fingerprinting), helping differentiate legitimate devices from slow-rate attackers. The DDoS detection relies on analyzing the inherent behavior of each type of DDoS attacks. However, the study is based on assumptions and dynamic thresholds, which may lead to false positives and negatively impact legitimate users. For example, the detection mechanism for slow-rate attacks is proposed to be implemented when a victim is almost out of resources, which is not exclusive to attacks. Additionally, authors stand out that dynamic anomaly thresholds can affect legitimate users with slow connections.

Finally, Kapourchali et al. [24] proposed P4httpGuard, a mechanism to detect slow-rate attacks. P4httpGuard leverages P4 programmable switches to monitor HTTP packets and calculate the difference in number between incoming and outgoing HTTP packets. The P4 switches also calculate a total of 33 flow features from incoming packets. These features are obtained using CIC-flowMeter which is based on the 5-tuple: source IP, destination IP, source port, destination port, and protocol [25]. If the difference between the number of incoming and outgoing packets exceeds a certain threshold, the P4 switches send the features extracted from the incoming packets to the control plane. The control plane includes an ONOS controller that manages the network and serves as communication path between the P4 switches and an external detection module [26]. The ONOS controller forwards the features collected from the P4 switches to the detection module, which classifies the features and sends the results back to the ONOS controller. Based on the results of the external module, the ONOS controller installs rules in the P4 switches to block attacks. P4httpGuard uses KNN, RF, decision tree (DT) or multilayer perceptron (MLP) to classify malicious flows, achieving an F1-score of up to 94% and an accuracy of 91%. The authors also propose a mitigation strategy in which packets from malicious sources are dropped according to their MAC address. However, P4httpGuard was evaluated in a topology with a single network segment, which is unrealistic, since real traffic traverses several networks before reaching a destination. Moreover, blocking malicious sources based on their MAC address may affect the effectiveness of P4httpGuard, as the MAC address changes with each network segment hop. While the MAC address may remain consistent in an SDN, it will change if P4httpGuard is implemented in an SDN interconnected with traditional networks.

Table 1 compares our study with the works reported in this section. We mark the columns(ML and DL) based on the models they explore in their research. We present works based on the models they explore: ML if they train and assess any machine learning model, and for DL we follow the same criteria. Most of the studies reveal a gap and opportunity for the implementation of a security framework addressing slow-rate attacks, particularly in terms of scalability and implementation. Some of the key points are as follows: (i) DL models are little explored in previous works [10,14,15,18,22], which have proven to be better than ML in certain scenarios, such as cybersecurity or image processing with higher accuracy and lower pre-processing requirements [27]; (ii) Previous works implement small unrealistic networks for testing, with some authors considering topologies that include only two hosts and one switch [14,15,17,20], and therefore, their scalability remains unaddressed; (iii) Some works rely on external devices like CPUs [14,22], which reduces scalability and increases costs due to the need for additional components that are not supported by all devices; and (iv) other works also reveal an opportunity to implement protection systems that mitigate slow-rate attacks, in addition to detection. Additionally, only one prior study addresses slow-rate attacks and focuses on IoT devices. However, unlike our work, this study does not consider the use of ML or DL for attack detection.

3. Background

This section provides background knowledge of slow-rate DDoS attacks and programmable networks.

3.1. Iot and slow-rate DDoS

The exponential growth of IoT devices, the data they generated, and their applications in various sectors has brought about several challenges in their development, including complexity, scalability, power consumption, and security [28–30]. In terms of security, IoT devices are often connected to the internet, making them vulnerable to hacking, which can lead to serious security breaches. This is particularly concerning when IoT devices are used in critical systems such as healthcare and transportation.

When designing protection mechanisms, IoT devices are often overlooked. This is partly due to the lack of standardization, given the vast number of devices from different manufacturers and their limited processing power, which prevents the implementation of complex security systems [7]. However, recent DDoS attacks are taking into account IoT devices, as they lack protection but are increasing in numbers in recent networks. Notable examples include the botnets created by Mirai and Hajime, which significantly amplify the power of DDoS attacks [12].

Slow-rate DDoS attacks, which are emerging as a serious cybersecurity threat, represent a new type of attack that particularly exploits IoT devices [9,12]. Unlike flood DDoS attacks, slow-rate attacks operate at the application layer and require minimal processing power, storage, and bandwidth. Remarkably, only a limited number of IoT devices are capable of establishing slow-rate connections that can disrupt critical services. Adding to the complexity, attackers can be mistaken for benign traffic coming from hosts with low bandwidth, making detection and mitigation particularly challenging [10].

3.2. SDN and programmable data plane

IoT devices are typically deployed in large numbers and generate vast amounts of data. Managing and orchestrating these devices can be complex and challenging. SDN technology has proven effective in addressing these challenges faced by IoT systems [31].

While in traditional networking the control and data planes are attached to network devices such as switches, in the SDN paradigm, the control plane is detached from the switches and installed in a logically centralized controller [32]. With this configuration, additional functions can be developed in a simpler manner as information is available through the controller, and SDN allows modular architectures with dynamic routing and global monitoring capabilities [32].

In terms of network security, attack detection and mitigation applications are typically deployed in the SDN controller. However, the controller must constantly monitor the state of the network to gather the necessary data. Not all data are available in the control plane e.g., network packets. Therefore, the control plane can rely on external devices, such as switches, to collect additional information increasing latency [11]. Furthermore, the SDN controller often handles multiple applications simultaneously, which can create additional overhead, degrading the response times and performance of critical security applications if a single controller instance is used [11]. For example, network monitoring can be deployed at the control plane, however, developing network monitoring without impact in network performance presents challenges. Continuous monitoring would cause additional overhead, but a lack of monitoring would cause incorrect behavior in management applications [33]. A distributed control plane can be leveraged to increase the performance of the SDN control plane. Nevertheless, the use of distributed control plane presents further challenges, including the need for communication channels, the possibility of conflicting flow rules, varying requirements for each controller instance and inconsistencies due to changes in the network state [11]. Additionally, some network information, such as packet headers, is only available at the data plane and must be passed to the controller, further complicating the process. To address these challenges, we propose a hybrid framework that fosters collaboration between the control plane and data plane. This approach aims to reduce the overhead of the control plane by delegating certain tasks to the data plane.

P4 is a programming language designed to define how switches process incoming packets [34]. P4 can be used to implement protocols already included in traditional switches or code custom applications to create new functions that would otherwise require hardware intervention. P4 operates with match-action tables. Match-action tables define one or more actions, which are executed according to one or several keys. The keys can be packet header fields or custom keys defined in P4. For instance, P4 can implement routing by using IP addresses as keys for forwarding actions, with the rules in the tables being populated by the control plane. It is possible to create functions that operate with little intervention from the control plane, which reduces latency as the data plane may not need to constantly communicate with the control plane and wait for a response. For instance, it is possible to build an autonomous ARP responder in P4, in which the control plane only fills the match-action tables and no other process is required.

P4 enables the creation of custom applications that run locally on the switch. The programmer can rewrite and recompile the code to fix errors or introduce new features without hardware intervention.

An important limitation of P4 is that it does not support complex functions, such as loops or floating-point calculations. This restriction exists because P4 applications are designed to run in linear time. Floating-point operations require several processor actions, and loops increase the time complexity of programs running on the switch [35].

4. System architecture

Our architecture is designed to detect Slowloris attacks by leveraging switches programmable with the P4 language for feature calculation, alongside an IDS and an IPS for classification and mitigation. The IDS and IPS are further explained in SubSections 4.2 and 4.3, respectively. Fig. 1 depicts the architecture proposed to detect and mitigate Slowloris DDoS in IoT networks. In this architecture, a border P4 switch is used to perform in-line packet processing for feature calculation. Features are then used by the IDS and IPS for attack detection and mitigation, respectively. In a typical IoT network setup, this switch can be positioned in front of the gateway to detect attacks before malicious packets reach critical infrastructure.

As shown in Fig. 1, the P4 switch captures network flows by processing packets in real-time. Flow features are identified using a two-tuple ID consisting on the source IP address and destination IP address (1). These features are then encapsulated in an IPv4 packet header (2), and sent to the IDS via a cloned packet (3). The IDS preprocesses the received header and uses ML or DL models to classify flows (4). Finally, if an attack is detected, the intrusion protection system (IPS) inserts mitigation rules in the P4 match-action tables to block the malicious flows (5). The process is further described in Algorithm 1 for the data plane and in Algorithm 2 for the control plane operations.

The process in the data plane, as outlined in Algorithm 1, involves calculating features for inference and an index for storing their values in switch registers. A packet with its TCP FIN flag set to 1 indicates the end of a flow. We choose FIN flags to determine the end of a flow. The start of a flow is marked by the arrival of a packet with a new pair of {IPSrc, IPDst}. When a new flow arrives, the programmable switches send the features of the last flow to the control plane by appending them to a cloned packet with a custom header and reset the values in the registers.

Algorithm 1 Network flow processing with P4 switches.

Input: `Packet (Ethernet/IP/TCP)`
for each inbound `Packet` **do**
  $\vec{F} = calculateFeatures(\texttt{Packet})$ ▷ Calculate Features
  $I = hash(\texttt{Packet.Ethernet.IP})$ ▷ Obtain Register index
  $R_{arrays} = saveFeatures(\vec{F}, I)$ ▷ Save features in registers
  if Packet.TCP.FINFlag == 1 **then**
    `ClonedPacket` = `Packet` ▷ Clone incoming packet
    `ClonedPacket` = `ClonedPacket` $\bigcup \vec{F}$ ▷ Append features
    $ControlPlane \leftarrow$ `ClonedPacket` ▷ Send to the control plane
    $R_{arrays} = resetRegisters(I)$ ▷ Reset the registers in index
  end if
end for

The process in the control plane described in Algorithm 2 that requires a cloned packet, the mean (M) and standard deviation (STD) values of the features (pre-loaded values) as inputs. The mean and standard deviation values are obtained during dataset preprocessing and saved in CSV files for feature normalization. Cloned packets received as input are the packets sent by P4 switches. The control plane operates by receiving the features calculated by the data plane and extracting them from the cloned packet. The control plane then normalizes the features and uses them to classify the flow by using a pre-trained model. Counters are initialized as empty to keep track of each attack flow detected for each flow identifier. If an attack is confirmed, the control plane writes rules to the programmable data plane to block the attackers by dropping their incoming packets.

Algorithm 2 Attack detection and mitigation with the control plane

Input: $ClonedPacket(Ethernet/IP/TCP/Custom), M, STD$
  $\vec{Attack} = []$ ▷ Initialize attack array
  for each inbound `ClonedPacket` **do**
    $\vec{F}$ = `ClonedPacket.Custom` ▷ Extract features
    $\vec{NF} = (\vec{F} - M)/STD$ ▷ Normalize features
    $C = classify(\vec{NF})$ ▷ Classify features, 0: normal, 1: attack
    if $C == 1$ **then** ▷ Confirm attack
      $\vec{Attack}(IP) = \vec{Attack}(IP) + 1$
    else if $C == 0$ & $\vec{Attack}(IP) > 0$ **then**
      $\vec{Attack}(IP) = \vec{Attack}(IP) - 1$
    end if
    if $\vec{Attack}(IP) >= 3$ **then**
      $DataPlane \leftarrow dropRule(IP)$ ▷ Modify match-action tables
    end if
  end for

The following sub-sections elaborate on the design of each component of the architecture presented in Fig. 1.

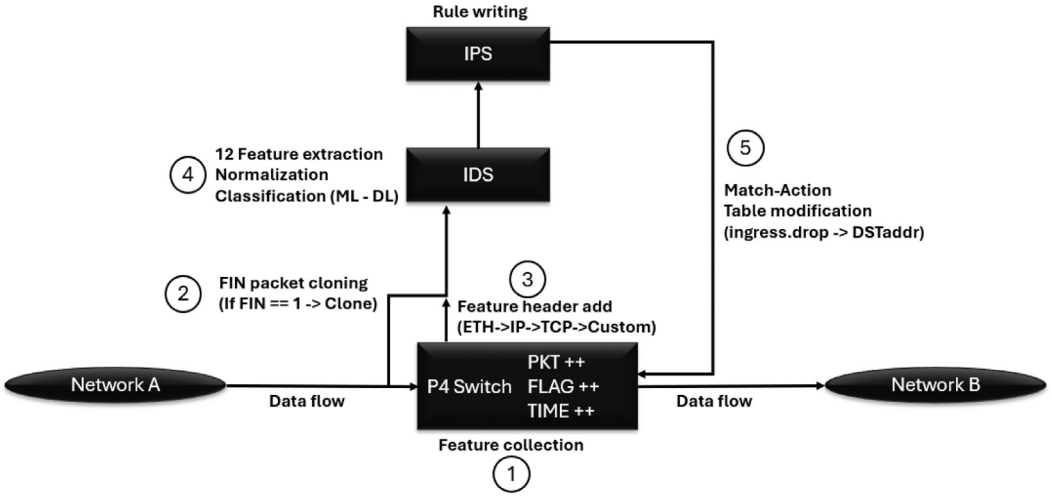


Fig. 1. Proposed architecture in frontier switches.

4.1. In-line packet processing with P4

The proposed attack detection and mitigation depends on pre-trained detection models. However, detection models require information to classify and thus detect attacks. In-line packet processing is performed to calculate the required features to allow the detection models to detect an attack. The P4 switch calculates 12 features (see Table 2) by processing each incoming packet to capture IPv4 and TCP header fields, including the source IP (IPSrc), destination IP (IPDst), TCP flags (SYN, ACK, FIN, URG, PSH, and RST) and size of the packet (bits). Additionally, the switch uses P4 artifacts such as timestamps to calculate additional features, including inter arrival times (IAT). P4 also provides artifacts and structures to aid in the operation of custom applications. An example of such structures is registers. Registers in P4 are memory locations that store information for subsequent packet processing. Each register has a specific entry size and each entry can be accessed by its index. In our implementation, one register is defined for each feature calculated in the switch.

When a packet enters the P4 switch, the switch calculates features of the flow to which the packet belongs to with the pair {IPSrc, IPDst}. Based on the header parameters, the features are saved in registers according to a hash between the fields {IPSrc, IPDst}. The flow features and the size of the registers are shown in Table 2. The size is important because the packet header used to send the features to the IDS must be a multiple of eight in size. Moreover, feature size must match the register size. Every new IP tuple requires additional space in the registers. As more features are added, more registers and memory are required. In our framework, the size of the feature is based on the maximum value found in the dataset for each feature. For example, the size of the URG count is smaller than the others because its maximum value found in the dataset requires fewer bits to be represented.

As shown in Table 2, the features can be calculated using simple arithmetic and comparison operations. This guarantees minimal delay introduced in the in-line packet processing as it ensures that the features can be calculated in linear time. For each flow, a packet containing a FIN flag marks the end of the flow. A threshold T for the maximum duration of the flow is considered to make the system resistant to memory overflow. Each new flow information, formed as ((IPSrc, IPDst): {Features}), is sent to the IDS via a cloned packet with a custom header. After this operation, the registers that store the flow features are reset to avoid memory overflow.

Alternative methodologies for flow delineation, such as using a fixed packet count or a time window, were analyzed. While effective for high-rate DDoS attacks, these methods fall short in accurately capturing the malicious behaviors of our specific targeted attack. A size window or packet count, effective for DDoS, may not be feasible for slow-rate due to the behavior of these attacks. In slow-rate DDoS scenarios, adversaries aim to minimize the packet count per connection to avoid detection and prolong the connection lifespan with the victim server. During hyperparameters tuning, other flow delineations were explored, but the method using FIN flags resulted in the highest inference accuracy. Therefore, our simplified flow definition, using the TCP-IP protocol, was shown to be the optimal approach for capturing the dynamics of Slowloris attacks at both the packet and flow levels.

We limit our scope to TCP packets to be able to calculate the 12 features mentioned in Table 2. Additionally, a selection of time segments or packets would fall in an arbitrary use of a flow delimiter, as there is a broad amount of choices. Instead we select FIN packets to follow a correct delineation already included in TCP communications. As slow-rate attacks establish a legitimate communication with the victims, the use of TCP packets can help in detecting such attacks [10].

The features calculated are based on Neto et al. [13], chosen for their effectiveness in classifying attacks and for their alignment with features used in the related works. The selected features adhere to the critical requirement of being calculable in P4, avoiding floating point operations or loops. More complex features would require additional operations that would necessitate external devices, raising scalability concerns as the framework would then rely on more devices, which may not be feasible to implement in every network.

Table 2
List of flow features. The flow ID is represented by the pair {IPSrc, IPDst}. The size of each feature in bits is also shown and is the size needed to allocate the largest record of each feature. This size is needed in the packet header to deliver the information for classification.

Feature	Description	Size (bits)
Duration [s]	Duration of the flow.	48.0
Packet count	Number of packets.	14.0
ACK count	Number of ACK flags.	14.0
PSH count	Number of PSH flags.	14.0
RST count	Number of RST flags.	14.0
URG count	Number of URG flags.	10.0
SYN count	Number of SYN flags.	14.0
Min packet IAT [s]	Minimum value of {Times between packets within a flow}.	48.0
Max packet IAT [s]	Maximum value of {Times between packets within the flow}.	48.0
Length Min	Minimum value of {Size in bits of the packets within the flow}.	8.0
Length Max	Maximum value of {Size in bits of the packets within the flow}.	16.0
Flow IAT [s]	Time between current and previous flows with the same ID.	48.0

4.2. Intrusion detection system (IDS)

The IDS, connected to the P4 switch, receives packets containing network flow information. The IDS, connected to P4 switches, runs a python script containing the feature extractor and classifier. Python Scapy library [36] is utilized to sniff and capture data from the incoming packets to the IDS. Once the IDS extracts the flow features, these features are normalized. The detection models used for the IDS are trained offline and saved in files for later use in Mininet environment deployment (see Section 5.2). The IDS loads the file of a previously saved detection model and performs classification employing the normalized features. We choose the highest accuracy achieved by the models as our criteria for their deployment during the experiments.

The classifiers evaluated in this work are KNN, DT, RF, long short-term memory (LSTM), CNN, gated recurrent unit (GRU), and MLP. These classifiers were selected due to their demonstrated effectiveness in detecting network attacks, as seen in related work. KNN, RF and CNN were the classifiers with wider use in the related work, but we also complemented these classifiers with the ones from Yungaicela-Naula et al. [37]. Approaches with higher accuracy are better to address slow-rate attacks to reduce the impact on legitimate users; thus, based on our results, robust classifiers such as RF are preferred. However, if the environment requires a fast classifier, DT or KNN can be used instead. Researchers that aim to implement our framework can use results obtained in Section 5.1 to select the most suitable model for each environment considering the testing time and accuracy. The output of the IDS ((IPSrc, IPDst): {malicious/benign}) is sent to the IPS.

4.3. Intrusion protection system (IPS)

To mitigate detected attacks, we propose a collaboration between the control plane and data plane to block attacks according to the classification results of a detection model using a python script. The IPS runs in the same host as the IDS. The IDS sends the classification result to the IPS along with the IPSrc and IPDst. If a malicious flow is detected, the IPSrc is added to a suspect list, and a counter is initiated to 1. If the next flow from the same source is classified as suspicious, the counter increases; if it is classified as benign, the counter decreases. If the counter reaches 0, the IPSrc is removed from the suspect list. However, if the counter for a specific IPSrc reaches 3, the script uses P4Utils [38] to modify the match-action tables in the P4 switch, blocking the IPSrc.

The P4 program defines a mitigation table specifically for dropping packets associated with an IPSrc. In our implementation, both the detection and mitigation strategies are executed on the same external device.

5. Experiments and results

This section details the experiments and results for the proposed framework. First, a set of ML and DL models is evaluated using a benchmark dataset. From this analysis, only a subset of the best models is selected to be assessed using simulations.

5.1. ML and DL model training

The experiments regarding model training were conducted offline using Jupyter notebooks. The system used was a Windows 11 laptop with 8 × 1.8 GHz cores and 32 GB of RAM.

The first step in the process was to identify an appropriate set of features for training ML and DL models to effectively detect Slowloris attacks. To this end, the CICIoT2023 dataset from the Canadian Institute for Cybersecurity was used [13].

The CICIoT2023 dataset contains traffic traces collected over several hours, featuring a topology of 105 IoT devices. It includes 33 different types of attacks, including high-rate DDoS attacks and one slow-rate DDoS attack. For the purpose of this study, we focused on the Slowloris DDoS attack, which is the only slow-rate attack in the dataset. The Slowloris attack exploits timeout vulnerabilities characteristic of slow-rate attacks [39]. The authors of the dataset provided the code for model training, feature calculation, PCAP files and preprocessed CSV files. Specifically, the CSV files contain flow features calculated using the DPKT Python library [13]. The dataset includes one PCAP file for Slowloris traffic (labeled SlowLoris) and four files for benign traffic (labeled BenignTraffic, BenignTraffic1, BenignTraffic2 and BenignTraffic3). A PCAP file is a type of file used to store captured network packets over a time span.

Table 3
Samples extracted offline using Python with a split of 80% for model training and 20% for testing. Samples for both classes come from the CICIoT2023 dataset.

File	Number of flows
Slowloris	41 437
Benign	81 490

Table 4
ML and DL model performance using the CICIoT2023 dataset.

Model	Accuracy (%)	Training time (s)	Testing time (s)
KNN	96.34%	0.34	2.96
RF	97.02%	82.86	1.73
DT	96.06%	0.22	0.01
LSTM	95.76%	85.77	1.83
CNN	95.77%	52.79	1.71
GRU	95.80%	119.92	2.72
MLP	95.42%	23.97	1.40

Due to the limited operations allowed by P4, we could not directly use the provided code for CSV files containing flow features in our implementation. The CSV files provided by Neto et al. [13] contain features calculated through means and standard deviations and thus are unable to represent features calculated in P4. Instead, we generated our own CSV files and calculated features from the PCAP files using Pandas, and DPKT libraries with Python. For training and testing, we used BenignTraffic and SlowLoris PCAP files. We also saved the mean and standard deviation of each feature in the dataset for subsequent use during experiments. During experiments, features are normalized using the mean and standard deviation before a prediction model performs classification.

The total number of flows used for training is 41,437 for Slowloris traffic and 81,490 for benign traffic as shown in Table 3. We split the data in 80% for training and 20% for testing. Hyperparameters for the models were tuned using both random search and grid search in Python. Each model was trained 20 times with three folds, and the results were averaged over those trials.

Accuracy was selected as the primary performance metric as it is the most commonly reported metric in the related work. The accuracy is calculated as follows.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \tag{1}$$

where TP is True positives, TN is True negatives, FP is False Positives and FN is False Negatives.

Table 4 presents the performance metrics for each model, including the accuracy average training time, and average testing time. As shown in Table 4, ML models outperformed the DL models in terms of accuracy. This can be explained by the simplicity of the features, which were chosen to be efficiently calculated by P4. The fastest model is DT and has the third best accuracy, which may be a reasonable trade-off in a network environment where speed may be priority over performance [40]. Nevertheless, RF has overall the highest accuracy so it is reasonable to keep this model for further analysis. Among the DL models, accuracy remained above 95%, with differences of less than 1%. This suggests that while their detection performance is very similar, the key distinction lies in their speed. The CNN model exhibited

a balance between accuracy and speed, making it a suitable candidate for evaluation in future tests.

Ultimately, RF and CNN were selected to be tested using simulations. This analysis is presented in the next sub-section. We choose models with higher accuracy than inference speed given the nature of slow-rate attacks. It is of greater importance to have a higher accuracy when detecting slow-rate attacks. Moreover, missclassifications of slow-rate attacks can negatively affect legitimate user performance [10]. We believe that selecting models with higher accuracy is appropriate given the nature of the attacks we are studying.

5.2. Experimental results

In this section, the trained ML and DL models are assessed using simulations with open-source tools.

5.2.1. Experimental setup

The experiments were conducted in a simulated environment using Mininet, running on an Ubuntu virtual machine with 4 cores and 16 GB RAM. The P4 switch model used was the BMv2 while the Mininet hosts and switches are the ones implemented by Tomek [41]. Fig. 2 shows the topology deployed for the simulations.

The top network in Fig. 2 represents an infrastructure network with H10 and H11 representing IoT devices. The infrastructure network also contains the IDS and the switches S1 and S2, which are the devices that implement our proposed solution. The IDS runs a python script with the detection and mitigation strategies, while the switches S1 and S2 are P4 switches performing routing and feature calculation.

The bottom network in Fig. 2 consists of hosts H1, H2, H3, H4, H5 and H6 representing IoT devices, as well as several switches that are primarily used for routing. The hosts in this network represent devices may either be part of a botnet, executing attacks on the infrastructure network, or they may represent legitimate devices. In a real-world scenario, the victim would not be aware of which devices are involved in the attack.

5.2.2. Traffic generation

Testing the proposed framework requires generating two types of traffic: benign IoT traffic and Slowloris DDoS attack traffic. To the best of our knowledge, there are currently no tools specifically designed for IoT traffic generation, and general-purpose tools like Netcat or Iperf may not accurately represent IoT traffic. For generating benign IoT traffic, we inject traffic from the PCAP files provided by the datasets using Tcprewrite and Tcpreplay. For Slowloris traffic, we used the Slowloris tool from Gkbrk [39] targeting an Apache server as the victim.

5.2.3. Detection performance

We evaluated the performance of our detection models before proceeding to the final experiment. As illustrated in Fig. 2, hosts H1, H3 and H5 are responsible for performing attacks, while hosts H2, H4 and H6 inject benign traffic from PCAP files. Hosts H1, H2, H3, H4, H5 and H6 are IoT devices located in an external network. We capture the packets cloned from S1 and S2 using Scapy and save the features in CSV files using Pandas for later offline tests. The benign traffic is generated from unused Pcap files from the CICIoT2023. More benign traffic samples were added using unseen packets from the Aposemat IoT-23 [42] dataset. Slowloris traffic is generated using the Slowloris tool.

The P4 switches in the network are tasked with calculating flow features and forwarding them to the control plane for further processing. The features arriving at the control plane can be stored in files and used to perform model fine tuning.

Table 5 displays the additional files used for further training and testing, along with their origin and the number of samples. Benign1, Benign2 and Benign3 are pcap files from the CICIoT2023, containing benign traffic data. The Echo PCAP file comes from the Aposemat IoT-23 dataset, also containing benign traffic data. The Slowloris1 and Slowloris2 files contain the features from flows generated using the Slowloris tool with different configurations.

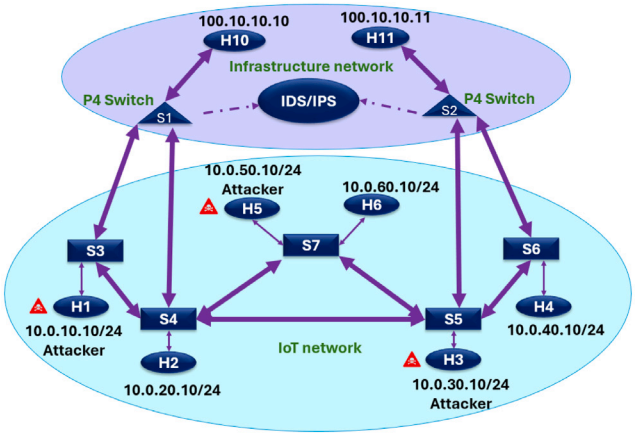


Fig. 2. Proposed topology for final experiments. The P4 switches are frontier switches between the infrastructure network and the internet.

Table 5
Additional data gathered to perform model fine tuning.

File	Samples	Origin	Use
Slowloris1	4618	Slowloris attack tool	Fine tuning
Benign3	7013	CICIoT2023	Fine tuning
Slowloris2	11 939	Slowloris attack tool	Testing
Benign1	12 899	CICIoT2023	Testing
Benign2	15 872	CICIoT2023	Testing
Echo	798	Aposemat	Testing

Table 6
RF and CNN performance using the experimental testbed.

Model	Accuracy	Recall		F1 score	
		Benign	Slowloris	Benign	Slowloris
RF	98.28%	97.57%	99.93%	98.76%	97.21%
CNN	98.11%	97.46%	99.62%	98.62%	96.99%

For the analysis of individual models, we combined the entire dataset and calculated additional metrics, as shown in Table 6. This table presents the final accuracy, along with two additional metrics for each class and are sensitive to false positives: Recall and F1-Score. These metrics are defined as follows.

$$Recall = \frac{TP}{P} = 1 - \frac{FN}{P}, \tag{2}$$

$$F1 - Score = \frac{2TP}{2TP + FP + FN} \tag{3}$$

where P is Positive (Benign), N is Negative (Slowloris), TP is True positives, TN is True negatives, FP is False positives, and FN is False negatives.

To demonstrate the performance of our framework, Figs. 3 and 4 present the confusion matrices for the RF and CNN models used in the final experiments.

Overall the detection accuracy for both models is above 98%, with a difference of less than 1%. However, when examining recall and F1-Score, the results show a distinct behavior. Recall indicates that the models are better at classifying Slowloris attacks than benign traffic since most of the attacks were correctly classified but benign traffic is being misclassified as attack. The false negatives from benign traffic can be observed in the F1-score, as those misclassified attacks reduce the F1-score for Slowloris as they are taken into the calculations. In summary, both models perform with accuracy up to 98%, but some benign traffic flows are misclassified as attack traffic.

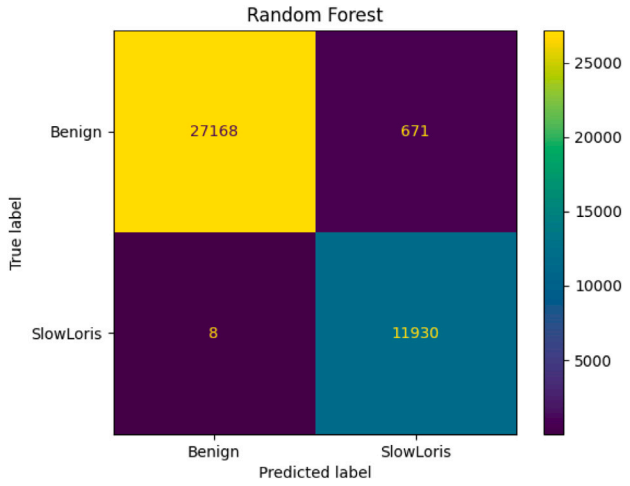


Fig. 3. Confusion matrix of the RF model.

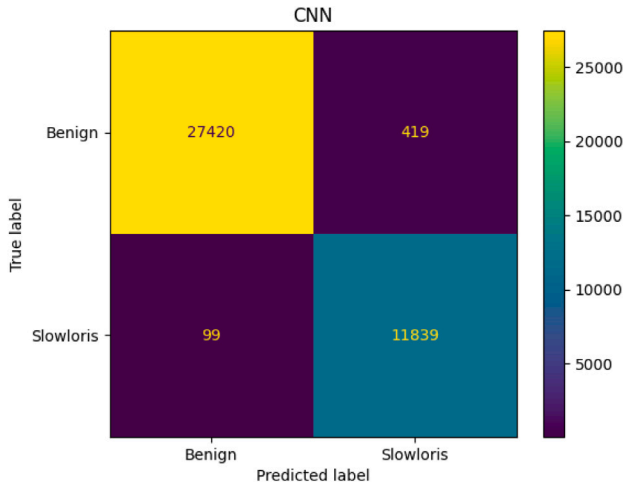


Fig. 4. Confusion matrix of the CNN model.

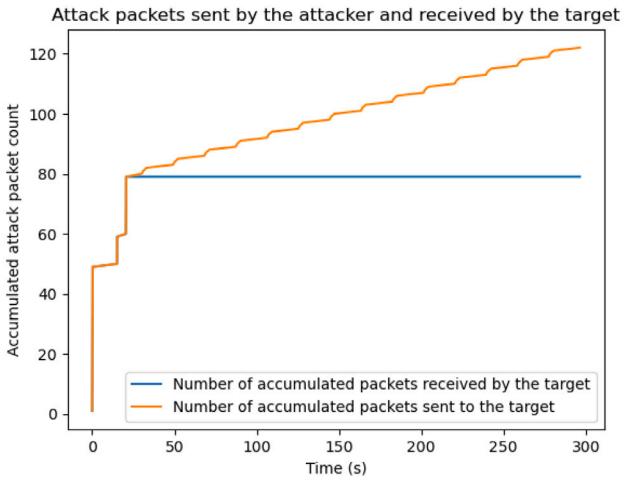


Fig. 5. Plot with the number of attack packets accumulated over the time. The packets sent by the attacker highlighted in orange and the received by the victim highlighted in blue. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

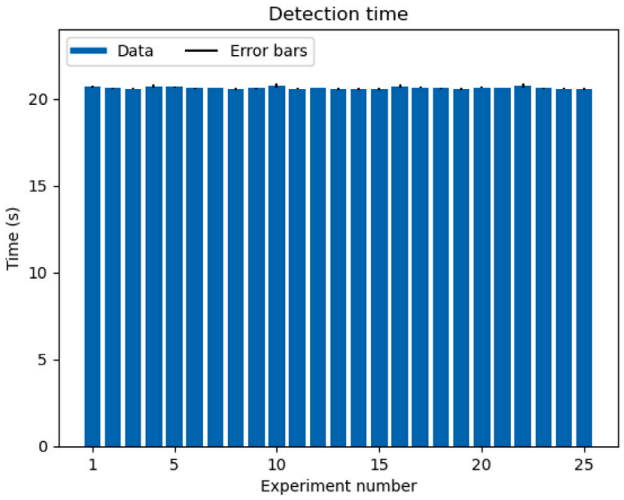


Fig. 6. Plot and confidence interval of the detection time during 25 experiments.

5.2.4. Mitigation performance

Finally, the framework was evaluated by conducting several multiple attacks using hosts 10 and 11 along with benign traffic injection in Fig. 2, with the CNN model deployed in the controller. The IDS successfully detected the attacks and performed mitigation by modifying the match-action tables in the P4 switches. Due to the effectiveness of the proposed framework, botnets performing Slowloris attacks could not establish communication with the hosts in the infrastructure network.

Fig. 5 illustrates the attacker sending Slowloris traffic over 5 min and the victim receiving the attack traffic. The figure shows an incremental plot of attack packets, it reflects the total number of packets received during the attack. The victim stops receiving attack traffic from the attacker after roughly 21 s and thus the blue line flattens, as no more attack packets were received by the victim after the mitigation takes action. On the opposite, the orange line increases as the attacker attempts new attacks but fails. We count three consecutive attack flows to reduce false positives and ensuring that packets from legitimate users are not dropped. For instance, the probability of detection (recall) of a benign flow for RF is 97.57%, leaving a miss rate of 2.43%. The probability of misclassify all three flows in a row is 0.0014%, which is much smaller than the miss rate. To further analyze the results, the python scripts were modified to record timestamps when an attack starts and when detection or mitigation is performed. A total of 25 experiments were performed to measure the detection and mitigation times. The corresponding plots are shown in Figs. 6 and 7, for the detection and mitigation times respectively along with the error bars lines. The bars represent the time measurement of each experiment and the lines present the confidence interval as error bars.

6. Discussion

Despite the limitations in loops or floating point operations of P4, the attack detection performed up to 98% in accuracy with only 12 features calculated from the IP and TCP headers. We designed and implemented a framework with acceptable accuracy in detecting and mitigating Slowloris attacks. Compared to the work from Kapourchali et al. [24], our framework achieves higher accuracy in detecting Slowloris attacks and also considers a scalable topology with IoT devices. Additionally, our framework obtains an accuracy similar to the works that mitigate flood DDoS. However, it is well known that slow-rate attacks are harder to detect than volumetric attacks [5,10,37,43]. Therefore, we believe that a framework capable of detecting Slowloris attacks with high accuracy has the potential to perform better against volumetric DDoS attacks as those presented in the related work. Prior studies have shown accuracy rates of approximately 95% or lower for

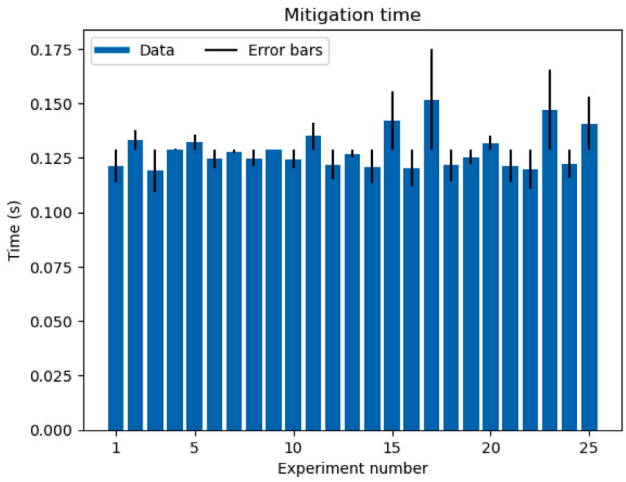


Fig. 7. Plot and confidence interval of the mitigation time during 25 experiments.

slow-rate attack detection, especially when tested in real-world testbeds rather than solely through dataset-based analysis. Therefore we set an accuracy of 95% as an acceptable reference point to achieve higher results than those in the literature, as our work is evaluated in a testbed [24,37].

We measured the detection and mitigation response times. On average, our system detects an attack in 20.629 s and mitigates it in 0.128 s. The mitigation response time measures the elapsed time between the effective detection of an attack and the mitigation strategy performed by the control plane.

The relatively long detection time is expected given the nature of slow-rate attacks. Unlike flood attacks, which can generate a large volumes of data even in milliseconds, slow-rate attacks can occur with minimal packet transmission over extended periods. However, the framework was able to detect and mitigate attacks after observing just three consecutive malicious flows, with mitigation time remaining minimal in comparison to detection time.

Our framework's design, including flow definition and captured features, is tailored to the characteristics of slowrate attacks. Even though our experiments focused in Slowloris attacks, we believe our solution can be effectively applied to the broader slow-rate attack family with comparable performance. We have identified that most slow-rate DDoS attacks follow similar patterns from the network perspective. In this context, a trained ML with one type of slow-rate attack was able to detect another type of slow rate attack with similar performance (fine-tuning might be needed). Therefore, our framework and methodology can be generalized to the slow-rate DDoS attack family [24,37,44–46].

In Table 7, we provide a comparison in terms of performance and implementation between our solution and previous works. As seen in this table, our study provides valuable insights on Slowloris DDoS attacks in IoT environments, which have not been sufficiently explored in previous works. For instance, we can delegate feature calculation to the SDN data plane and successfully detect Slowloris attacks by leveraging ML and DL. Previous researchers have focused on flood DDoS attacks, which are easier to detect due to their substantial packet volume. Therefore, we consider that the performance that our framework achieved for Slowloris DDoS detection is significant, as it is comparable to the achieved by previous works for flood DDoS attacks. Moreover, we surpassed the work from Kapourchali et al. [24] in accuracy.

In terms of implementation, most previous works do not perform mitigation or rely on feature calculation to external devices instead of leveraging a programmable data plane. However, relying on external devices to perform more operations creates scalability and costs problems when the network grows. Our work is the only one that detects and mitigates Slowloris attacks by leveraging P4 programmable switches, achieving even higher results than some of the flood detection frameworks.

By offloading feature calculation to a programmable data plane, our system can scale to more complex networks. This advantage is further amplified when leveraging the capacity of P4 to implement custom functions. This capability lowers the need for specialized devices. In our implementation we were capable of developing a packet analyzer, relieving the control plane from feature calculation tasks. To give a broader vision regarding scalability of our work, Table 7 includes a lists of which actions are performed at P4 switches or rely on external devices. Our work only relies on external devices to perform actions regarding attack detection and mitigation.

7. Conclusion and future work

Slow-rate DDoS have the potential to efficiently use IoT devices to attack critical networks. In this paper we implemented a mitigation framework against Slowloris attacks with the help of P4 and either ML or DL to detect Slowloris DDoS in IoT networks. We explored 3 DL models and 3 ML models with an accuracy up to 98% during tests using two recent datasets CICDDoS2023 and a portion of Aposemat IoT-23. Furthermore, our solution was tested using Mininet, Python and BMv2 P4 switch were the framework successfully detected and mitigated Slowloris attacks. Our study not only explored an attack insufficiently reviewed in literature, but it also brings alternatives for IoT protection using SDN by leveraging programmable switches and P4 and performing mitigation without limiting to detection.

In future work, we plan to train the models to detect additional slow-rate DDoS attacks, and test our framework in a physical implementation.

We are also interested on implementing a ML or DL model directly in P4. If we are able to find a simple model such as decision tree, then it is possible to escalate our implementation to perform detection directly in the switch, with the only limitation of performing mitigation externally.

CRediT authorship contribution statement

Erick D. Ramirez-Martinez: Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Jesús A. Pérez-Díaz:** Writing – review & editing, Supervision, Methodology, Funding acquisition, Conceptualization. **Noe M. Yungaicela-Naula:** Writing – review & editing, Methodology, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by FRIDA (Fondo Regional para la Innovación Digital en America Latina *y* el Caribe), award number 6856.

The first author is deeply thankful with the support offered by the scholarships from Tecnologico de Monterrey and National Council of Science and Technology of Mexico (CONACYT).

Data availability

Data will be made available on request.

Table 7
Performance from different implementations in literature. This table lists the accuracy achieved from several solutions in literature and the actions performed by P4 switches or external devices.

Author	Flood	Slow	P4 Switch	External devices
Carvalho et al. [14]	97.94%	–	Information collection	Preprocessing, Normalization, Model training, Detection
Macias et al. [15]	96.00%	–	Information collection	Feature calculation, Detection
Musumeci et al. [16]	98.00%	–	Feature calculation	Detection
Roshani and Nobakht [17]	99.00%	–	Packet cloning, Feature calculation	Detection
Sadi et al. [21]	97.00%	–	Flow monitoring	Feature calculation, Detection, Mitigation
Mecerhed et al. [18]	98.95%	–	Information collection	Feature calculation, Detection
Friday et al. [10]	NA	NA	Flow monitoring, Information collection, Classification, Mitigation	Detection
Quintero González et al. [22]	95.00%	–	Flow monitoring, Packet cloning, Mitigation	Feature calculation, Detection
Doriguzzi-Corin et al. [20]	99.87%	–	Bloom filtering, Feature calculation	Normalization, Detection
Ma et al. [23]	99.21%	–	Trusted Node Monitoring, Rerouting	Flow Monitoring, Feature calculation, Detection, Mitigation
Kapourchali et al. [24]	–	91.00%	HTTP packet tracking, Feature calculation	Detection, Mitigation
Our work	–	98.00%	Feature calculation	Normalization, Detection, Mitigation

References

[1] I. Lella, C. Ciobanu, E. Tsekmezoglou, M. Theocharidou, E. Magonara, A. Malatras, R. Svetozarov Naydenov, ENISA Threat Landscape 2023: July 2022 to June 2023, Technical Report, European Union Agency for Cybersecurity (ENISA), 2023.

[2] P. Kumari, A.K. Jain, A comprehensive study of DDoS attacks over IoT network and their countermeasures, Comput. Secur. 127 (2023) http://dx.doi.org/10.1016/j.cose.2023.103096.

[3] M.M. Najafabadi, T.M. Khoshgoftaar, A. Napolitano, C. Wheelus, RUDY attack: Detection at the network level and its important features, in: Proceedings of the 29th International Florida Artificial Intelligence Research Society Conference, FLAIRS 2016, 2016, pp. 282–287.

[4] K. Doshi, Y. Yilmaz, S. Uludag, Timely detection and mitigation of stealthy DDoS attacks via IoT networks, IEEE Trans. Dependable Secur. Comput. PP (2021) 1, http://dx.doi.org/10.1109/TDSC.2021.3049942.

[5] D. Tang, R. Dai, Y. Yan, K. Li, W. Liang, Z. Qin, When SDN meets low-rate threats: A survey of attacks and countermeasures in programmable networks, ACM Comput. Surv. (2024) http://dx.doi.org/10.1145/3704434, Just Accepted.

[6] Imperva, The Imperva Global DDoS Threat Landscape Report 2023, Technical Report, Imperva, 2023.

[7] Microsoft, Microsoft Digital Defence Report 2022, Technical Report, Microsoft, 2022.

[8] S. Herwig, K. Harvey, G. Hughey, R. Roberts, D. Levin, Measurement and analysis of Hajime, a peer-to-peer IoT botnet, 2019, http://dx.doi.org/10.14722/ndss.2019.23488.

[9] S. Chaudhary, P.K. Mishra, DDoS attacks in Industrial IoT: A survey, Comput. Netw. 236 (2023) 110015, http://dx.doi.org/10.1016/j.comnet.2023.110015.

[10] K. Friday, E. Bou-Harb, J. Crichigno, M. Scanlon, N. Beebe, Offloading Network Forensic Analytics to Programmable Data Plane Switches, 2023, pp. 139–190, http://dx.doi.org/10.1142/9789811273209_0005.

[11] R. Deb, S. Roy, A comprehensive survey of vulnerability and information security in SDN, Comput. Netw. 206 (2022) 108802, http://dx.doi.org/10.1016/j.comnet.2022.108802, URL: https://www.sciencedirect.com/science/article/pii/S1389128622000299.

[12] Q. Li, H. Huang, R. Li, J. Lv, Z. Yuan, L. Ma, Y. Han, Y. Jiang, A comprehensive survey on DDoS defense systems: New trends and challenges, Comput. Netw. 233 (2023) http://dx.doi.org/10.1016/j.comnet.2023.109895.

[13] E.C.P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A.A. Ghorbani, CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment, Sensors 23 (13) (2023) http://dx.doi.org/10.3390/s23135941.

[14] R.N. Carvalho, L.R. Costa, J.L. Bordim, E.A.P. Alchieri, Detecting DDoS attacks on SDN data plane with machine learning, in: Proceedings - 2021 9th International Symposium on Computing and Networking Workshops, CANDARW 2021, 2021, pp. 138–144, http://dx.doi.org/10.1109/CANDARW53999.2021.00030.

[15] S.G. Macias, L.P. Gaspary, J.F. Botero, ORACLE: An architecture for collaboration of data and control planes to detect DDoS attacks, in: Proceedings of the IM 2021 - 2021 IFIP/IEEE International Symposium on Integrated Network Management, 2021, pp. 962–967.

[16] F. Musumeci, A.C. Fidanci, F. Paolucci, F. Cugini, M. Tornatore, Machine-learning-enabled DDoS attacks detection in P4 programmable networks, J. Netw. Syst. Manage. 30 (1) (2022) http://dx.doi.org/10.1007/s10922-021-09633-5.

[17] M. Roshani, M. Nobakht, HybridDAD: Detecting DDoS flooding attack using machine learning with programmable switches, 2022, http://dx.doi.org/10.1145/3538969.3538991,

[18] F. Mecerhed, A. Benabdallah, K. Zeraoulia, C. Benzaid, 3D-PM: A ML-powered probabilistic detection of DDoS attacks in P4 switches, in: 2023 IEEE International Mediterranean Conference on Communications and Networking, MeditCom 2023, 2023, pp. 80–85, http://dx.doi.org/10.1109/MeditCom58224.2023.10266635.

[19] E. Zardini, A. Delilbasic, E. Blanzieri, G. Cavallaro, D. Pastorello, Local binary and multiclass SVMs trained on a quantum annealer, IEEE Trans. Quantum Eng. 5 (2024) 1–12, http://dx.doi.org/10.1109/TQE.2024.3475875.

[20] R. Doriguzzi-Corin, L.A.D. Knob, L. Mendozzi, D. Siracusa, M. Savi, Introducing packet-level analysis in programmable data planes to advance Network Intrusion Detection, Comput. Netw. 239 (2024) http://dx.doi.org/10.1016/j.comnet.2023.110162.

[21] A.A. Sadi, M. Savi, D. Berardi, A. Melis, M. Prandini, F. Callegati, Real-time pipeline reconfiguration of P4 programmable switches to efficiently detect and mitigate DDoS attacks, in: Proceedings of the 26th Conference on Innovation in Clouds, Internet and Networks, ICIN 2023, 2023, pp. 21–23, http://dx.doi.org/10.1109/ICIN56760.2023.10073501.

[22] L.A. Quintero González, L. Castanheira, J.A. Marques, A.E. Schaeffer-Filho, L. Paschoal Gaspary, Bungee-ML: A cross-plane approach for a collaborative defense against DDoS attacks, J. Netw. Syst. Manage. 31 (4) (2023) http://dx.doi.org/10.1007/s10922-023-09769-6.

[23] J. Ma, W. Su, Y. Li, Y. Peng, Synchronizing DDoS detection and mitigation based graph learning with programmable data plane, SDN, Future Gener. Comput. Syst. 154 (2024) 206–218, http://dx.doi.org/10.1016/j.future.2023.12.033.
[24] R.F. Kapourchali, R. Mohammadi, M. Nassiri, P4httpGuard: detection and prevention of slow-rate DDoS attacks using machine learning techniques in P4 switch, Cluster Comput.-J. Netw. Softw. Tools Appl. 27 (6) (2024) 8047–8064, http://dx.doi.org/10.1007/s10586-024-04407-5.
[25] A.H. Lashkari, G. Drapper, M.S. Islam, Cicflowmeter, 2016, https://github.com/ahlashkari/CICFlowMeter. (Accessed 10 February 2025).
[26] ONF, Open network operating system (ONOS®), 2024, https://opennetworking.org/onos/. (Accessed 14 March 2024).
[27] L. Alzubaidi, J. Zhang, A.J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma, J. Santamaría, M.A. Fadhel, M. Al-Amidie, L. Farhan, Review of deep learning: concepts, CNN architectures, challenges, applications, future directions, J. Big Data 8 (1) (2021) 53.
[28] M.S. Virat, S. Bindu, B. Aishwarya, B. Dhanush, M.R. Kounte, Security and privacy challenges in internet of things, in: 2018 2nd International Conference on Trends in Electronics and Informatics, ICOEI, 2018, pp. 454–460, http://dx.doi.org/10.1109/ICOEI.2018.8553919.
[29] K. Iwanicki, A distributed systems perspective on industrial IoT, in: 2018 IEEE 38th International Conference on Distributed Computing Systems, ICDCS, 2018, pp. 1164–1170, http://dx.doi.org/10.1109/ICDCS.2018.00116.
[30] M. Mahamat, G. Jaber, A. Bouabdallah, Achieving efficient energy-aware security in IoT networks: a survey of recent solutions and research challenges, Wirel. Netw. 29 (2) (2023) 787–808.
[31] S. Siddiqui, S. Hameed, S.A. Shah, I. Ahmad, A. Aneiba, D. Draheim, S. Dustdar, Toward software-defined networking-based IoT frameworks: A systematic literature review, taxonomy, open challenges and prospects, IEEE Access 10 (2022) 70850–70901, http://dx.doi.org/10.1109/ACCESS.2022.3188311.
[32] I.A. Valdovinos, J.A. Pérez-Díaz, K.-K.R. Choo, J.F. Botero, Emerging DDoS attack detection and mitigation strategies in software-defined networks: Taxonomy, challenges and future directions, J. Netw. Comput. Appl. 187 (2021) 103093, http://dx.doi.org/10.1016/j.jnca.2021.103093, URL: https://www.sciencedirect.com/science/article/pii/S1084804521001156.
[33] F. Bannour, S. Souihi, A. Mellouk, Distributed SDN control: Survey, taxonomy and challenges, IEEE Commun. Surv. Tutor. PP (2017) 1, http://dx.doi.org/10.1109/COMST.2017.2782482.
[34] ONF, P4 open source programming language, 2024, https://p4.org/. (Accessed 14 March 2024).
[35] M. Avanzini, U. Dal Lago, G. Moser, Analysing the complexity of functional programs: higher-order meets first-order, SIGPLAN Not. 50 (9) (2015) 152–164, http://dx.doi.org/10.1145/2858949.2784753.
[36] Scapy, Welcome to Scapy's documentation!, 2024, https://scapy.readthedocs.io/en/latest/. (Accessed 21 March 2024).
[37] N.M. Yungaicela-Naula, C. Vargas-Rosales, J.A. Perez-Diaz, SDN-based architecture for transport and application layer DDoS attack detection by using machine and deep learning, IEEE Access 9 (2021) 108495–108512, http://dx.doi.org/10.1109/ACCESS.2021.3101650.
[38] nsg-ethz, P4-utils, 2023, https://github.com/nsg-ethz/p4-utils. (Accessed 21 March 2024).
[39] Gkbrk, Slowloris.py - Simple slowloris in Python, 2023, https://github.com/gkbrk/slowloris. (Accessed 28 February 2024).
[40] T. Lin, D. Liu, X. Jiuqiang, Research on reliable transmission based on UDP in industrial SDN, 2023, pp. 727–743, http://dx.doi.org/10.1007/978-981-99-2730-2_69.
[41] O. Tomek, Network prototyping made easy with P4 and Python!, 2018, https://osinstom.github.io/en/network-prototyping-p4/#. (Accessed 28 February 2024).
[42] S. Garcia, A. Parmisano, M.J. Erquiaga, IoT-23: A labeled dataset with malicious and benign IoT network traffic, 2021, http://dx.doi.org/10.5281/zenodo.4743746.
[43] A.A. Alashhab, M.S.M. Zahid, M. Abdullahi, M.S. Rahman, Real-time detection of low-rate DDoS attacks in SDN-based networks using online machine learning model, in: 2023 7th Cyber Security in Networking Conference, CSNet, IEEE, 2023, pp. 95–101.
[44] N.M. Yungaicela-Naula, C. Vargas-Rosales, J.A. Perez-Diaz, D.F. Carrera, A flexible SDN-based framework for slow-rate DDoS attack mitigation by reinforcement, J. Netw. Comput. Appl. 205 (2022) http://dx.doi.org/10.1016/j.jnca.2022.103444.
[45] N.M. Yungaicela-Naula, C. Vargas-Rosales, J.A. Perez-Diaz, SDN/NFV-based framework for autonomous defense against slow-rate DDoS attacks by using reinforcement learning, Future Gener. Comput. Syst.-Int. J. Esci. 149 (2023) 637–649, http://dx.doi.org/10.1016/j.future.2023.08.007.
[46] N.M. Yungaicela-Naula, C. Vargas-Rosales, J.A. Perez-Diaz, E. Jacob, C. Martinez-Cagnazzo, Physical assessment of an SDN-based security framework for DDoS attack mitigation: Introducing the SDN-SlowRate-DDoS dataset, IEEE Access 11 (2023) 46820–46831, http://dx.doi.org/10.1109/ACCESS.2023.3274577.



Erick D. Ramírez-Martínez received the B.E. degree in Digital Systems and Robotics from Tecnologico de Monterrey, Mexico, in 2022. In 2023 he started a Master in Computer Science in Tecnologico de Monterrey where he is actively developing cybersecurity network protocols based in machine learning and deep learning. His research interests include the application of artificial intelligence strategies for cybersecurity.



JESÚS ARTURO PÉREZ-DÍAZ (Member, IEEE) received the B.Sc. degree in computer science from the Autonomous University of Aguascalientes, in 1995, and the Ph.D. degree in new advances in computer science systems from Universidad de Oviedo, in 2000. From 2000 to 2002, he became a Full Associate Professor with the University of Oviedo. He is currently a Researcher and a Professor with Tecnológico de Monterrey—Campus Guadalajara, Mexico, and a member of the Mexican Researchers National System. His research interests include cybersecurity in SDN and the design of communications protocols, where he has supervised several master's and Ph.D. theses in the field. He was recognized by the COIMBRA Group as one of the Best Young Latin-American Researchers, in 2006, and received a research stay with Louvain Le Nouveau University, Belgium. He has been awarded by the CIGRE and Intel for the development of innovative systems. He was a recipient of the Best Student Award for his B.Sc. degree from the Autonomous University of Aguascalientes.



NOE M. YUNGAICELA-NAULA received the Ph.D. degree in Engineering Sciences from Tecnologico de Monterrey, Mexico, in 2023. In 2023, he joined the Centre for Secure Information Technologies (CSIT) at Queen's University Belfast (QUB), Northern Ireland, UK, as a Research Fellow. His current research interests include the use of artificial intelligence techniques to automate different tasks in next-generation networks (SDN, NFV, 5G, and Open RAN), such as intrusion detection systems and network resource optimization.